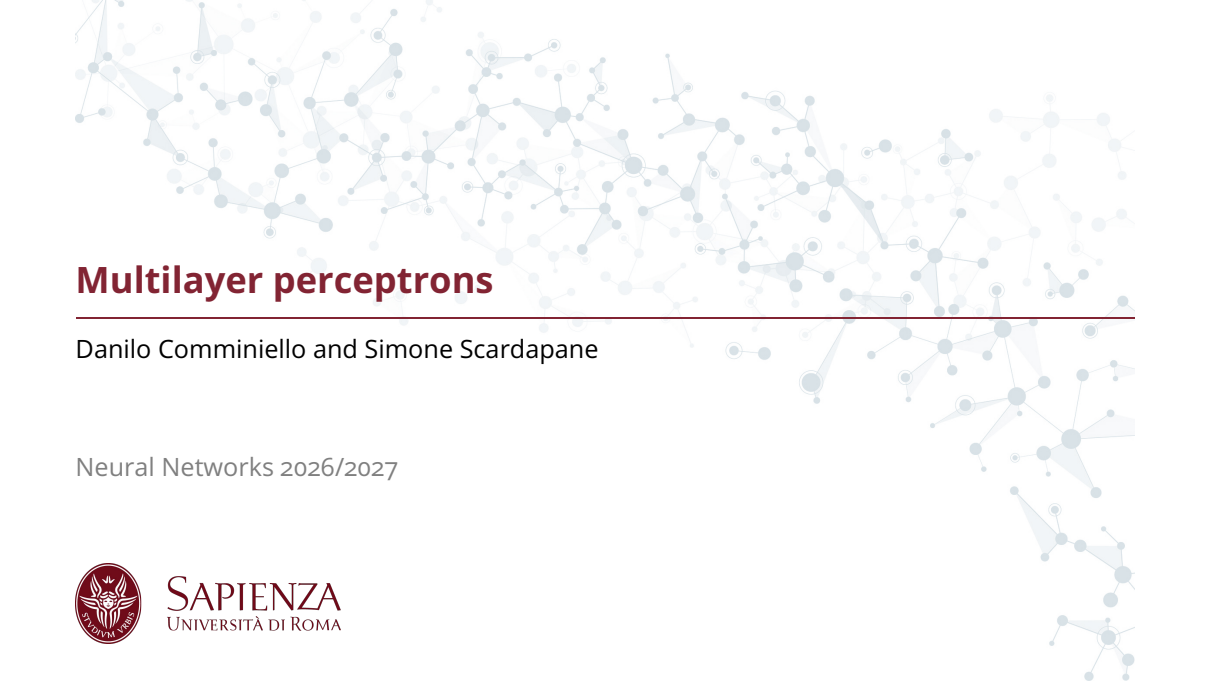




Multilayer perceptrons

Danilo Comminiello and Simone Scardapane

Neural Networks 2026/2027



SAPIENZA
UNIVERSITÀ DI ROMA

1 Feedforward neural networks

Basic definitions

Activation functions

Combining layers beyond the basic MLP

The multilayer perceptron

A **fully-connected neural network**, also known as a **multilayer perceptron (MLP)**, learns an intermediate representation of the input:

Hidden representation

$$\mathbf{h} = \phi(\mathbf{W}\mathbf{x} + \mathbf{b})$$

ϕ is an elementwise **activation function**.

Linear output

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{h} + c$$

A linear model applied to the learned features $\mathbf{h}$.

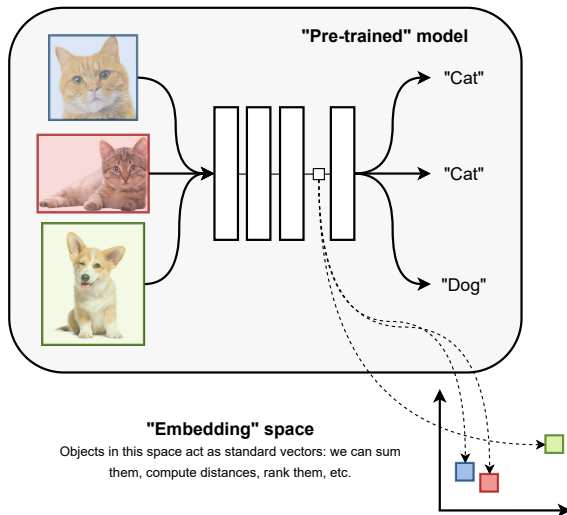
The parameters $\mathbf{W}$, $\mathbf{b}$, $\mathbf{w}$, c are learned jointly from the data.

Without the nonlinearity ($\phi(z) = z$), the two affine maps collapse:

$$f(\mathbf{x}) = \mathbf{w}^\top (\mathbf{W}\mathbf{x} + \mathbf{b}) + c = (\mathbf{w}^\top \mathbf{W})\mathbf{x} + (\mathbf{w}^\top \mathbf{b} + c).$$

Embedding the data

The hidden representation provides new coordinates for the input, in which a linear output layer can distinguish the classes.



Adding hidden layers

Nothing prevents us from adding additional **hidden** (intermediate) layers:

$$\begin{aligned}\mathbf{h}^{(0)} &= \mathbf{x}, \\ \mathbf{h}^{(\ell)} &= \phi_{\ell}(\mathbf{W}^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}), \quad \ell = 1, \dots, L, \\ f_{\theta}(\mathbf{x}) &= \mathbf{w}^{\top} \mathbf{h}^{(L)} + c.\end{aligned}$$

All weights and biases belong to $\theta = \{\mathbf{W}^{(\ell)}, \mathbf{b}^{(\ell)}\}_{\ell=1}^L \cup \{\mathbf{w}, c\}$.

For widths $D_0, \dots, D_L$ (D_0 is the input dimension),

$$\mathbf{W}^{(\ell)} \in \mathbb{R}^{D_{\ell} \times D_{\ell-1}}, \quad \mathbf{b}^{(\ell)} \in \mathbb{R}^{D_{\ell}}.$$

Layer ℓ has $D_{\ell}(D_{\ell-1} + 1)$ parameters; the scalar output adds $D_L + 1$.

Widths and activations are layer-specific **hyperparameters**.

Training the network

Given a dataset $\mathcal{S} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, we minimize:

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \frac{1}{N} \sum_{i=1}^N \ell(f_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i).$$

We use the losses introduced in the linear-model lecture, with the appropriate output for regression or classification (i.e., adding a softmax or sigmoid for multi-class or binary classification, respectively). In order to compute gradients we can use **automatic differentiation**.

Universal approximation

Let $g : K \rightarrow \mathbb{R}$ be continuous on a compact set $K \subset \mathbb{R}^d$, and let ϕ be continuous and non-polynomial.

For any $\varepsilon > 0$, there exists an MLP with **one hidden layer**, sufficiently large width, and biases such that:

$$\sup_{\mathbf{x} \in K} |g(\mathbf{x}) - f_{\theta}(\mathbf{x})| < \varepsilon.$$

This is an **existence result**. It does not tell us how much width or data we need, or whether training will find suitable parameters.

A single hidden layer can be sufficient for approximation; it need not be an efficient choice.

Leshno, Lin, Pinkus and Schocken (1993), Neural Networks, 6(6), 861–867.

Some terminology

Representation / embedding

The feature vector $\mathbf{h}^{(\ell)}$ computed at an intermediate layer.

Width and depth

Width: $\dim \mathbf{h}^{(\ell)}$. Depth: L hidden layers ($L + 1$ with the output).

Parameters: learned

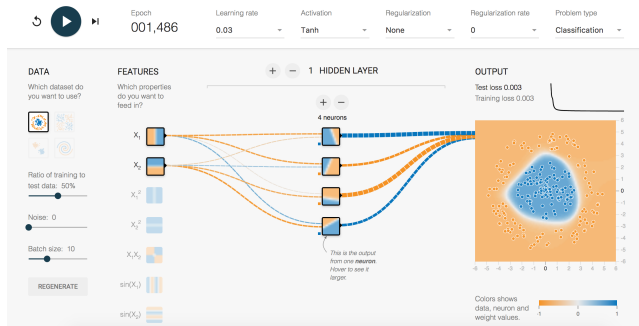
The weights and biases θ learned from the data.

Hyperparameters: chosen

Layer count, widths, activation functions, and learning rate.

Choosing hyperparameters is called **model selection** / **hyperparameter optimization**.

Playing with a neural network



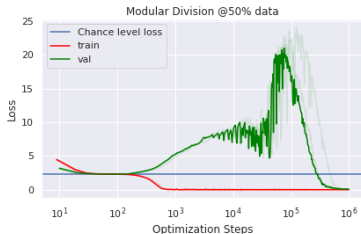
playground.tensorflow.org

Change the number and width of the hidden layers. How do the learned representations and the decision boundary change?

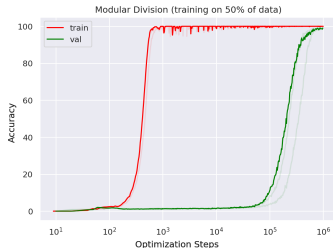
Training neural networks can be weird

Train a model (transformer, later in the course) to learn division modulo 97. What happens?

Loss over training



Accuracy in the same experiment



Definitely not intuitive: multiple descents for the validation loss, with accuracy improving long after perfect memorization (**grokking**).

Other surprising phenomena we have no time to cover: fitting random labels, double descent as model size grows, and low-loss paths connecting different solutions.

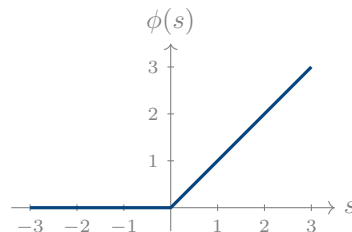
Original plots: Power et al. (2022), Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets, Figs. 1 and 4.

The rectified linear unit (ReLU)

The **rectified linear unit** is defined as:

$$\phi(s) = \max(0, s).$$

In words: negative inputs become zero, positive inputs pass through unchanged.



ReLU is a good default choice for MLPs: we will explain more in the automatic differentiation lecture, when we study how gradients propagate through the network.

One network, many affine maps

Each ReLU either passes its input or sets it to zero:

$$\mathbf{h} = \mathbf{D}(\mathbf{x})(\mathbf{W}\mathbf{x} + \mathbf{b}), \quad D_{ii}(\mathbf{x}) \in \{0, 1\}.$$

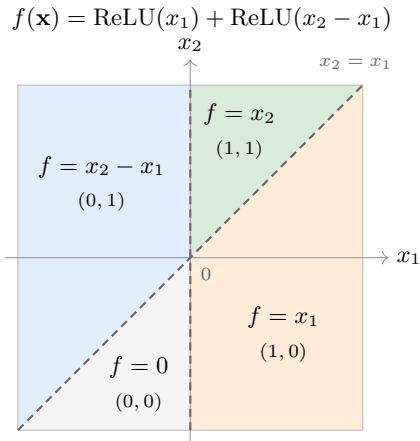
The diagonal entries of $\mathbf{D}$ indicate which units are active. With an affine output,

$$\begin{aligned} f(\mathbf{x}) &= \mathbf{w}^\top \mathbf{D}(\mathbf{x})(\mathbf{W}\mathbf{x} + \mathbf{b}) + c \\ &= \mathbf{a}(\mathbf{x})^\top \mathbf{x} + \beta(\mathbf{x}). \end{aligned}$$

Within each region, the gates stay fixed: the network is **exactly affine**.

This holds at any depth. Training learns both the regions and their affine maps.

For ReLU hidden layers, before any output sigmoid or softmax. Montúfar et al. (2014), Linear regions of deep neural networks.

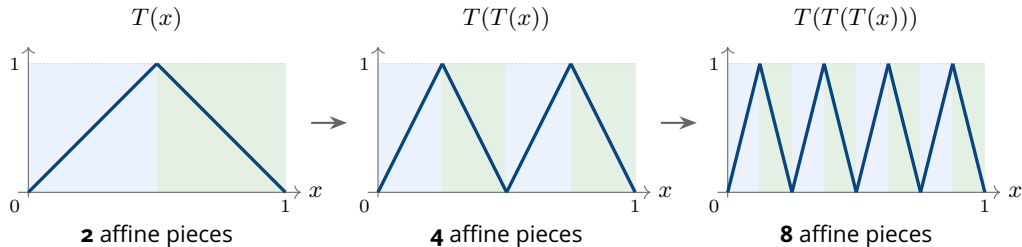


Labels: output formula and gate pattern (d_1, d_2)

Depth through composition

Three ReLUs make a triangular function. Apply the same function repeatedly:

$$T(x) = 2 \operatorname{ReLU}(x) - 4 \operatorname{ReLU}\left(x - \frac{1}{2}\right) + 2 \operatorname{ReLU}(x - 1), \quad x \in [0, 1].$$



Each composition splits every piece in two: k blocks give 2^k pieces using $3k$ ReLU units.

A single hidden layer with m ReLUs has at most $m + 1$ pieces in one dimension. **Depth can make a representation much more compact.**

Without nonlinearities, all layers collapse to one affine map.

An expressivity example, not a guarantee about training or generalization. Telgarsky (2016), Benefits of depth in neural networks.

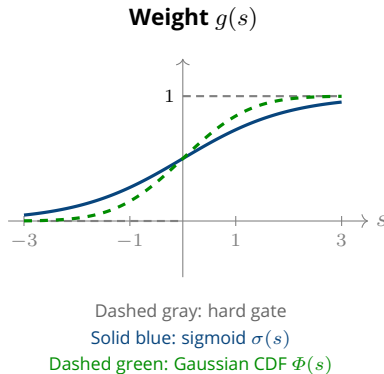
Smooth activation weights

Replace ReLU's binary gate with a **smooth weight**:

$$\phi(s) = s g(s), \quad 0 < g(s) < 1.$$

The sigmoid and Gaussian CDF give two common choices, leading to **SiLU** and **GELU**.

The resulting activations are smooth; the exact piecewise-affine description no longer applies in general.



SiLU and GELU

SiLU (Sigmoid-weighted linear unit):

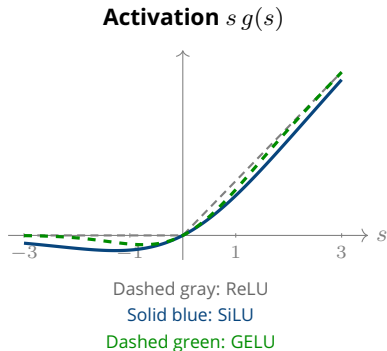
$$\text{SiLU}(s) = s \sigma(s), \quad \sigma(s) = \frac{1}{1 + e^{-s}}.$$

GELU (Gaussian error linear unit):

$$\text{GELU}(s) = s \Phi(s).$$

where Φ is the standard Gaussian cumulative distribution function.

Elfwing et al. (2017); Hendrycks and Gimpel (2016).



Learning the negative slope

Instead of fixing the whole activation, we can let training adjust its shape.

$$\phi_{\alpha}(s) = \begin{cases} s & s \geq 0, \\ \alpha s & s < 0. \end{cases}$$

LeakyReLU

Fix a small negative-side slope, e.g., $\alpha = 0.01$.

Parametric ReLU (PReLU)

Learn α jointly with the weights and biases.

For PReLU, α is part of θ . It can be shared or learned separately for different units.

He et al. (2015), Delving Deep into Rectifiers.

Learning the activation shape

Learn a more flexible shape using fixed **basis functions**:

$$\phi(s) = \sum_{k=1}^K \alpha_k \psi_k(s).$$

The coefficients α_k are trained jointly with the network.

Kernel activation functions (KAFs)

$$\psi_k(s) = \kappa(s, d_k), \quad \kappa(s, d_k) = \exp(-\gamma(s - d_k)^2).$$

Example: Gaussian bumps with fixed centers d_k and width parameter $\gamma > 0$.

Non-parametric: a flexible basis expansion, still implemented with finitely many parameters. Splines are another option.

Scardapane et al. (2019), Kafnets.

Kolmogorov–Arnold networks

We can also change where the learned scalar functions act:

MLP with learned activations

$$h_i = \phi_i \left(\sum_j W_{ij} x_j + b_i \right).$$

Mix inputs, then transform each unit.

KAN layer

$$h_i = \sum_j W_{ij} \phi_{ij}(x_j).$$

Transform each connection, then sum.

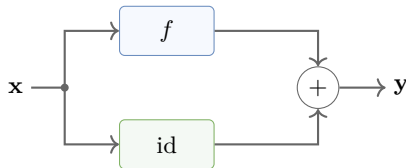
A KAN learns an activation at a unit, while a KAN learns functions on connections. Flexibility, however, comes with additional parameters and computation, making them less efficient than (equivalently deep) ReLU-based models.

Liu et al. (2024), KAN: Kolmogorov–Arnold Networks.

Combining branches

Two blocks can also process the same input in parallel. Given $\mathbf{u} = f(\mathbf{x})$ and $\mathbf{v} = g(\mathbf{x})$, we can combine their outputs in different ways:

- **Concatenation**, $[\mathbf{u}; \mathbf{v}]$, preserves both feature vectors and adds their widths.
- **Addition**, $\mathbf{u} + \mathbf{v}$, combines matching coordinates and requires equal shapes.
- Choosing the identity for one branch gives a **residual connection**: $\mathbf{y} = \mathbf{x} + f(\mathbf{x})$.



Concatenation example: DenseNet (2017); residual networks: He et al. (2016).

Input-dependent branch weights

Instead of adding branches with fixed coefficients, we can learn weights that depend on the input.

Highway connections

$$\mathbf{t} = \sigma(\mathbf{W}_t \mathbf{x} + \mathbf{b}_t),$$

$$\mathbf{y} = \mathbf{t} \odot f(\mathbf{x}) + (\mathbf{1} - \mathbf{t}) \odot \mathbf{x}.$$

Each gate coordinate lies in $[0, 1]$ and controls how much of the input is carried forward. Similar gates appear in recurrent models; $\odot$ denotes element-wise multiplication.

Mixtures of experts (MoEs)

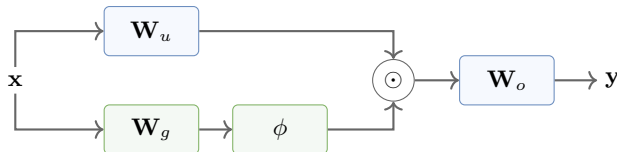
$$\mathbf{y} = \sum_{e=1}^E a_e(\mathbf{x}) f_e(\mathbf{x}).$$

A router weights the outputs of several experts with the same output shape. Sparse MoEs, used in language models, save computation by evaluating only the selected experts.

Srivastava et al., Highway Networks (2015); Shazeer et al., Sparsely-Gated MoEs (2017).

Multiplicative blocks: GLU and SwiGLU

We can also multiply two branch outputs elementwise. In a GLU block, one projection determines how the other is weighted:



$$\mathbf{y} = \mathbf{W}_o [(\mathbf{W}_u \mathbf{x}) \odot \phi(\mathbf{W}_g \mathbf{x})] .$$

With $\phi = \sigma$ we obtain a **GLU**, whose gate lies in $[0, 1]$. Replacing it with SiLU gives **SwiGLU**, used in Llama: its multiplier can also be negative or exceed one.

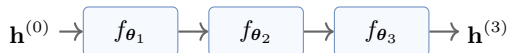
For $\mathbf{x} \in \mathbb{R}^D$, $\mathbf{W}_u, \mathbf{W}_g \in \mathbb{R}^{M \times D}$ and $\mathbf{W}_o \in \mathbb{R}^{D \times M}$. Biases are omitted.

Shazeer (2020); Touvron et al., LLaMA (2023).

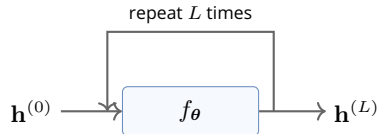
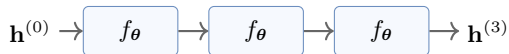
Sharing parameters and looping a block

A stack can use different parameters at each layer, or repeatedly apply the same block. The latter gives a **looped model**:

Separate parameters



Shared parameters



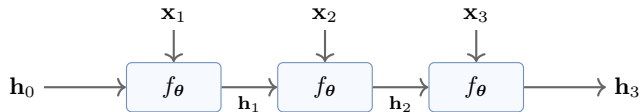
The shared block computes $\mathbf{h}^{(\ell+1)} = f_{\theta}(\mathbf{h}^{(\ell)})$, with matching input and output shapes. For a block with P parameters, sharing reduces the count from LP to P , but still requires L evaluations.

The unrolled example shows three applications. Dehghani et al., Universal Transformers (2019).

Recurrence over a sequence

If we supply a new input at each application, the shared block becomes a **recurrent model**:

$$\mathbf{h}_t = f_{\theta}(\mathbf{h}_{t-1}, \mathbf{x}_t).$$



The hidden state carries information from earlier inputs, while the same parameters are used throughout the sequence. The model can therefore process sequences of different lengths.

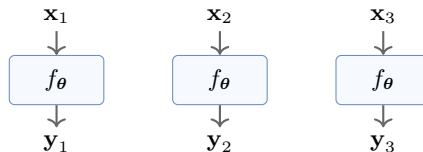
Unlike a loop over depth, this recurrence advances through the input sequence at each step.

Shared MLPs on structured data

An MLP can process each position of a sequence of vectors $\mathbf{x}_1, \dots, \mathbf{x}_T \in \mathbb{R}^D$ independently, using the same parameters:

$$\mathbf{y}_t = f_{\theta}(\mathbf{x}_t), \quad t = 1, \dots, T.$$

This mixes features within a position, but does not exchange information between positions.



The same idea applies to image pixels or patch vectors. Flattening the whole image instead allows global mixing, but ties the weights to a fixed layout.

Transformers use these **positionwise MLPs** alongside attention. Communication between positions requires another operation, such as convolution over neighborhoods, attention, or a recurrent state.